

Opracowanie

System e-commerce sklepu odzieżowego

Kacper Gumulak & Damian Judka

6.06.2026

Część 4: Implementacja Prototypu Systemu

4.1 Wybór technologii i koncepcja środowiska

Implementacja systemu opiera się na architekturze "grubego serwera bazy danych" (ang. thick database), gdzie przeważająca część krytycznej logiki biznesowej została zaimplementowana bezpośrednio w warstwie składowania danych. Stos technologiczny został dobrany tak, aby zapewnić maksymalną wydajność, bezpieczeństwo i niezawodność operacji biznesowych.

- **Baza Danych (Backend):** Głównym silnikiem systemu jest MariaDB 11. Wykorzystano w nim mechanizm pamięci masowej InnoDB, który gwarantuje pełną zgodność z właściwościami ACID (Atomowość, Spójność, Izolacja, Trwałość) dla wszystkich transakcji e-commerce. Sama logika biznesowa zajmuje około 2600 linii kodu SQL, obejmując widoki, funkcje i zaawansowane procedury składowane.
- **Wzorce Projektowe:** Wdrożono architekturę Event Sourcing dla logów magazynowych. Zamiast standardowego nadpisywania rekordów w tabelach produktowych, system zapisuje każde zdarzenie powiązane z ruchem magazynowym (tabela stock_events), a aktualny stan dostępności asortymentu jest dynamicznie wyliczany (widok v_inventory).
- **Interfejs Użytkownika (Prototyp Frontend):** Do zaprezentowania funkcjonalności bazy danych wykorzystano język Python oraz framework Streamlit. Wybór ten pozwolił na błyskawiczne wdrożenie responsywnego interfejsu graficznego, który działa jako lekki pośrednik, wywołujący procedury składowane i prezentujący wyniki w czasie rzeczywistym.
- **Koncepcja Środowiska Wdrożeniowego:** Zastosowano pełną konteneryzację środowiska za pomocą narzędzia Docker Compose. Infrastruktura została zoptymalizowana pod kątem bezpieczeństwa – zastosowano mechanizm

zmiennych środowiskowych, całkowicie eliminując ryzyko wycieku hardcodowanych poświadczeń z repozytorium kodu (np. danych logowania do bazy czy konfiguracji narzędzi administracyjnych).

4.2 Opis zaimplementowanego prototypu

Zbudowany prototyp w pełni pokrywa główny przypadek użycia systemu ("Kompleksowa realizacja zamówienia klienta"), weryfikując poprawność założeń architektonicznych opracowanych w poprzednich fazach projektu.

Aplikacja demonstruje podział ról, udostępniając dedykowane widoki dla Klienta oraz Administratora. Z poziomu interfejsu klienckiego użytkownik ma możliwość płynnego przeglądania asortymentu (zasilanego przez hierarchiczne struktury typu Nested Set) oraz zarządzania wirtualnym koszykiem. Weryfikacja dostępności ilościowej produktów w magazynie odbywa się w pełni asynchronicznie, chroniąc przed zjawiskiem "oversellingu" już na etapie dodawania do koszyka.

Najważniejszym elementem zaimplementowanego prototypu jest wysoki stopień automatyzacji przepływu pracy. Realizacja procedury `pay_order` z poziomu interfejsu nie tylko uaktualnia status zamówienia, ale natychmiastowo, wewnątrz pojedynczej transakcji bazodanowej, generuje docelową fakturę (tabela `invoices`) oraz przypisane do niej linie rozliczeniowe, odciążając tym samym warstwę aplikacji webowej.

4.3 Scenariusz działania głównej funkcjonalności

Prototyp umożliwia przeprowadzenie pełnej ścieżki e-commerce, która obejmuje następujące kroki:

1. **Ścieżka Zakupowa:** Wybór odzieży z uwzględnieniem atrybutów, umieszczenie w koszyku, podsumowanie finansowe oraz wprowadzenie danych wysyłkowych i rozliczeniowych.
2. **Realizacja Płatności:** Symulacja zatwierdzenia wpłaty skutkująca pełnym, automatycznym wygenerowaniem dokumentacji księgowej w bazie.
3. **Logistyka i Magazyn:** Przekazanie obsługi zamówienia na stronę Administracyjną. Użytkownik z uprawnieniami przygotowuje przesyłkę do wysyłki (wygenerowanie linku do śledzenia) i zmienia jej status na wysłaną.
4. **Weryfikacja Event Sourcing:** Wywołanie operacji zwrotu towaru w celu empirycznego zademonstrowania, w jaki sposób system dopisuje nowy rekord w dzienniku logów magazynowych i aktualizuje ogólną dostępność towarów bez ingerencji administratora w same stany liczbowe.